

DAY 12

1. Positional Parameters

What are Positional Parameters?

Positional parameters are special shell parameters used to access arguments passed to a script from the command line.

Suppose we have:

```
./script.sh apple 25 report.txt
```

Bash automatically assigns positions:

```
$1 → apple  
$2 → 25  
$3 → report.txt
```

So inside `script.sh`:

```
echo "$1"  
echo "$2"  
echo "$3"
```

would access those arguments.

Mental model

```
./script.sh apple 25 report.txt  
           |      |      |  
           ▼      ▼      ▼  
          $1     $2     $3
```

The numbering starts at **1**.

2. Why Are They Called "Positional"?

Because the meaning depends on the **position** of the argument.

For:

```
./script.sh Sourya 21 India
```

we have:

```
$1 = Sourya  
$2 = 21  
$3 = India
```

But if the user runs:

```
./script.sh India 21 Sourya
```

then:

```
$1 = India  
$2 = 21  
$3 = Sourya
```

The shell doesn't know that `$1` is supposed to be a name.

It simply means:

the first positional argument.

3. Arguments 1-9

For the first nine arguments, you can directly use:

```
$1  
$2  
$3  
...  
$9
```

Example:

```
./script.sh a b c d e f g h i
```

gives:

```
$1 = a
$2 = b
$3 = c
$4 = d
$5 = e
$6 = f
$7 = g
$8 = h
$9 = i
```

4. Arguments 10 and Beyond

This is a very important Bash rule.

For arguments beyond `$9`, use braces:

```
${10}
${11}
${12}
```

Why?

Consider:

```
$10
```

Bash can interpret this as:

```
$1 + 0
```

rather than:

```
argument number 10
```

Therefore:

```
${10}
```

clearly tells Bash:

```
"I want positional parameter number 10."
```

Rule to remember

```
$1 ... $9      → direct form is fine  
${10} onward  → use braces
```

5. Example with 10 Arguments

Suppose:

```
./script.sh a b c d e f g h i j
```

Then:

```
$1 → a  
$2 → b  
$3 → c  
$4 → d  
$5 → e  
$6 → f  
$7 → g  
$8 → h  
$9 → i  
${10} → j
```

6. Positional Parameters Are Read-Only Inputs

Positional parameters represent the arguments supplied to the script.

For example:

```
./calculator.sh + 10 20
```

The script can interpret:

```
$1 → +  
$2 → 10  
$3 → 20
```

This allows the same script to work with different inputs.

7. Practical Example – Calculator Script

The instructor's calculator example demonstrates a useful concept.

Suppose we execute:

```
./calc.sh + 10 20
```

The arguments are:

```
$1 = +  
$2 = 10  
$3 = 20
```

The script can use `$1` as the operator and `$2`, `$3` as numbers.

Conceptually:

```
./calc.sh + 10 20  
  |   |   |  
  ▼   ▼   ▼  
$1 $2 $3  
 |   |   |  
 |   └───┘ numbers  
 |  
 └───┘ operator
```

This demonstrates why positional parameters are useful for command-line tools.

8. Default Values with `${parameter:-default}`

Another very useful Bash feature is **parameter expansion with a default value**.

Syntax:

```
${parameter:-default}
```

Meaning:

Use the parameter's value if it is set and non-empty; otherwise use default .

Example:

```
name=${1:-Guest}
```

If the user runs:

```
./script.sh Sourya
```

then:

```
$1 = Sourya  
name = Sourya
```

But if the user runs:

```
./script.sh
```

then `$1` is unset, so:

```
name = Guest
```

9. Why Default Values Are Useful

Without a default:

```
name=$1
```

the variable may be empty when the user doesn't provide an argument.

With:

```
name=${1:-Guest}
```

the script has a fallback.

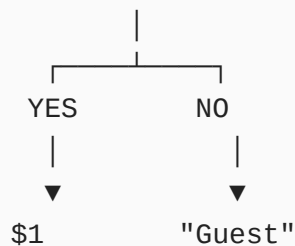
This is useful for:

- optional arguments
- configuration values
- scripts with sensible defaults
- preventing empty values from causing unexpected behavior

Mental model

```
${1:-Guest}
```

\$1 exists and isn't empty?



10. Special Parameters

Special parameters are predefined by the shell.

They provide information about:

- the script
- its arguments
- its execution context
- processes and shell state

The two parameters introduced here are:

```
$#  
$0
```

11. `$#` – Number of Arguments

`$#` represents:

The total number of positional parameters supplied to the script.

Example:

```
./script.sh apple banana orange
```

There are three arguments.

Therefore:

```
$# = 3
```

Another example:

```
./script.sh
```

There are no positional arguments:

```
$# = 0
```

12. Why `$#` Is Important

`$#` is extremely useful for **argument validation**.

Suppose your script requires exactly two arguments.

You can conceptually check:

```
if [ "$#" -ne 2 ]; then  
    ...  
fi
```

```
fi
```

Meaning:

```
| If the number of arguments isn't 2, show an error.
```

This prevents users from running the script incorrectly.

13. `$0` – Script Name

`$0` represents the name/path used to invoke the script.

Suppose you run:

```
./backup.sh file.txt
```

Then `$0` is typically:

```
./backup.sh
```

while:

```
$1 = file.txt
```

So:

```
$0 → script  
$1 → first argument  
$2 → second argument
```

Important distinction

```
$0 is not the first user argument.
```

The first positional argument is:

```
$1
```

14. \$0 for Usage Messages

This is one of the most useful applications of `$0`.

Suppose a script expects two arguments.

Instead of hardcoding:

```
Usage: ./script.sh <file> <directory>
```

you can use `$0`:

```
echo "Usage: $0 <file> <directory>"
```

If the user runs:

```
./backup.sh
```

the message can become:

```
Usage: ./backup.sh <file> <directory>
```

If the script was invoked differently, `$0` reflects that invocation name/path.

15. Combining \$# and \$0

This is a very common pattern.

Suppose a script requires **two arguments**.

Conceptually:

```
if [ "$#" -ne 2 ]; then
    echo "Usage: $0 <argument1> <argument2>"
    exit 1
fi
```

Here:

```
$# → checks how many arguments were supplied  
$0 → tells the user how to invoke the script correctly
```

For example:

```
./script.sh
```

could produce:

```
Usage: ./script.sh <argument1> <argument2>
```

16. Why Argument Validation Matters

Imagine a security script:

```
./scan.sh target output.txt
```

It expects:

```
$1 → target  
$2 → output file
```

If the user accidentally runs:

```
./scan.sh
```

the script may behave incorrectly.

Argument validation allows you to fail early:

```
Wrong number of arguments  
    ↓  
Show correct usage  
    ↓  
Exit
```

This makes scripts:

- safer
- easier to use
- easier to debug
- more predictable

18. `$0` vs `$1` vs `$#`

This is worth memorizing.

Suppose:

```
./script.sh apple banana orange
```

Then:

```
./script.sh apple banana orange
  |         |         |         |
  |         |         |         |
  ▼         ▼         ▼         ▼
  $0        $1        $2        $3

  $# = 3
```

So:

```
$0 → ./script.sh
$1 → apple
$2 → banana
$3 → orange
$# → 3
```

19. Common Beginner Mistakes

Mistake 1 – Thinking `$0` is the first argument

Wrong:

```
$0 = first argument
```

Correct:

```
$0 = script/invoke name  
$1 = first argument
```

Mistake 2 – Using \$10

Instead of:

```
echo "$10"
```

use:

```
echo "${10}"
```

Mistake 3 – Not validating arguments

Bad design:

```
input=$1  
output=$2
```

without checking whether the arguments were actually provided.

Better approach:

```
Check $# first  
  ↓  
If incorrect → show usage  
  ↓  
Otherwise → process arguments
```

Mistake 4 – Forgetting quotes

Prefer:

```
echo "$1"
```

rather than unnecessarily relying on:

```
echo $1
```

because argument values may contain spaces or wildcard characters.
